

Kobie

Architecture & Design Document

Autonomous Competitive Intelligence Agent for Loyalty Programs

Version 1.0

FastAPI

LangGraph

Gemini 2.5 Flash

Groq

SQLite WAL

Next.js 14

Pydantic v2

01 Project Overview

Kobie is an autonomous competitive-intelligence agent that researches customer loyalty programs (airline, hotel, retail, etc.) and produces analyst-grade, evidence-grounded briefs.

1 Input Resolution

LLM-backed identity validator resolves ambiguous program names via Gemini; max 3 clarification rounds.

2 Research Planning

Up to 15 targeted search queries planned across 10 source types (official, financial, review, forum, app_reviews, etc.).

3 Evidence Extraction

Schema-grounded facts per SCHEMA_FIELD_PATHS; per-field confidence, source URL, and EXTRACTED/NOT_FOUND/AMBIGUOUS status.

4 Conflict Adjudication

5-step ladder: identical-value short-circuit → confidence gap → field-type deterministic merge → adversarial debate.

5 Narrative Synthesis

600–900 word analyst brief from adjudicated FieldReport entries, organized by 8 schema categories.

6 Grounded Q&A;

Conversational interface strictly from final_brief + field_report; all answers traceable to extracted evidence.

Implementation: FastAPI backend (server.py) · LangGraph pipeline (pipeline/graph.py) · SQLite persistence (core/db.py) · Next.js 14 frontend (frontend/)

02 Problem Statement

Customer loyalty programs present a unique data challenge for competitive analysts. Information is distributed across official program pages, financial disclosures, travel blogs, forum threads, and app stores — each with varying accuracy and recency. Four core problems compound this difficulty:

Fragmented

No single authoritative source covers all program mechanics. Data spans official pages, app stores, financial filings, and user forums.

Volatile

Earn rates, tier thresholds, and point valuations change frequently. HIGH_VOLATILITY_FIELDS in core/schemas.py require recency-aware adjudication.

Contradictory

Official pages vs. blogs vs. forums frequently disagree on the same fact, requiring systematic conflict detection and resolution.

Expensive to Verify

Analysts must cross-reference many pages per program and adjudicate disagreements by hand — a process Kobie automates end-to-end.

Design constraint (README.md): Never use LLM training memory as a source of loyalty-program facts. Every supported claim must carry a source_url and be traceable to scraped content.

03 Objectives

1 Input Resolution

Resolve ambiguous/partial input to one canonical identity; max 3 clarification rounds, 5-min timeout each.

2 Research Planning

Plan and execute up to 15 targeted queries per program across 10 source types.

3 Evidence Extraction

Extract EXTRACTED/NOT_FOUND/AMBIGUOUS facts with per-field confidence and source attribution.

4 Conflict Resolution

Detect and resolve conflicts via cheapest-sufficient strategy ladder.

5 Brief + Q&A;

600–900 word analyst brief; grounded Q&A; traceable to field_report entries.

6 Comparison Mode

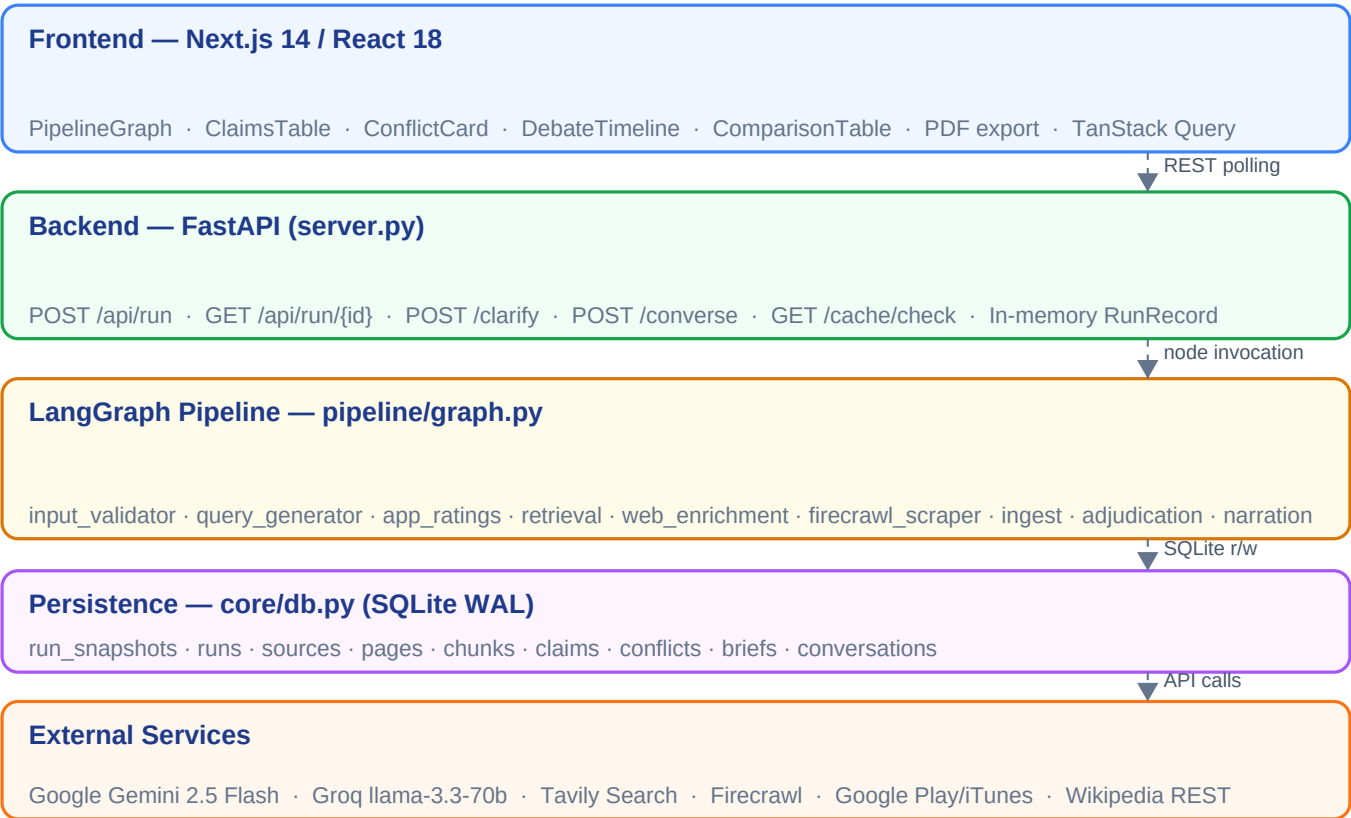
Side-by-side comparison with category-level verdicts and comparison brief.

7 Reliability & Caching

Track per-stage cost (tokens/USD), cache completed runs, survive process restarts.

04 High-Level Architecture

Architecture Layers



05 End-to-End Data Flow

1 Submission

User submits a loyalty program name via the Next.js frontend. POST /api/run creates a RunRecord in STORE with status=pending.

2 Validation

input_validator_node calls Gemini to resolve the program identity. Returns ValidationResult with status (resolved/ambiguous/rejected).

3 Cache Check

After validation, the pipeline checks run_snapshots for a prior completed run for the same canonical program. If found, the snapshot is replayed.

4 Query Generation

query_generator_node calls Gemini to plan up to 15 targeted search queries across 10 source types.

5 App Ratings Prefetch

app_ratings_node fetches Google Play Store and Apple iTunes ratings for the program app in parallel.

6 Retrieval

retrieval_node submits queries to Tavily Search, deduplicates URLs, and stores SourceRecord rows in the database.

7 Web Enrichment

web_enrichment_node fetches direct URLs and Wikipedia pages identified during query planning.

8 Firecrawl Scraping

firecrawl_node passes the top 20 URLs (by Tavily score) to the Firecrawl API for full-page markdown extraction.

9 Ingestion

ingest_node chunks all page content, calls Gemini to extract schema fields per chunk, and normalises raw claims into the claims table.

10 Adjudication

adjudication_node groups claims by field, detects conflicts, and resolves them via the 5-step cheapest-sufficient ladder.

11 Narration

narrator_node calls Gemini to produce a 600–900 word analyst brief from adjudicated FieldReport entries.

12 Persistence & Caching

All data is saved to SQLite. A run_snapshot is written for future cache hits.

13 Conversation

The user can ask questions via POST /converse. Answers are grounded strictly in final_brief + field_report.

14 Comparison Mode

POST /api/run with two programs triggers sequential pipeline runs, then a comparison brief with category-level verdicts.

06 System Components

Component	File(s)	Responsibility
FastAPI Server	server.py	HTTP API, RunRecord management, STORE dict, clarification/cache threading events
LangGraph Graph	pipeline/graph.py	Node registration, conditional edges, graph compilation; also direct node invocation for server.py
Input Validator	pipeline/stages/validation.py	Gemini-backed program identity resolution; clarification loop up to 3 rounds, 5-min timeout
Query Generator	pipeline/stages/query_generator.py	Gemini query planning; up to 15 queries across 10 source types with field_query_map
App Ratings	pipeline/stages/app_ratings_fetcher.py	Google Play Scraper + iTunes Search API; prefetch before Tavily to save budget
Retrieval	pipeline/stages/retrieval.py	Tavily Search; URL canonicalisation and deduplication into SourceRecord list
Web Enrichment	pipeline/stages/direct_url_seed_fetcher.py + wikipedia_fetcher.py	Seeds high-confidence direct URLs; Wikipedia summary — both non-fatal, zero Tavily budget
Firecrawl Scraper	pipeline/stages/firecrawl_scraper.py	Firecrawl API; round-robin top-20 URL selection; partial failure tolerant
Ingest Node	pipeline/nodes/ingest_node.py + chunker.py, extractor.py, normalizer.py	Raw store → chunk → Gemini extraction → normalise → FieldReport
Conflict Adjudicator	pipeline/adjudication/conflict_adjudicator.py	5-step resolution ladder; FIELD_STRATEGY_MAP; debate dispatch; FLAG → human_review_queue
Debate Engine	pipeline/adjudication/debate_engine.py	Groq advocate/judge calls; cosine similarity gate; key pool; 401 eviction
Narrator	pipeline/stages/narration.py	Gemini brief synthesis (600–900 words) from adjudicated FieldReport
Database Layer	core/db.py	SQLite WAL DDL (12 tables); connect/checkpoint; WAL-safe write lock
Schemas	core/schemas.py	AgentState TypedDict, Pydantic models, SCHEMA_FIELD_PATHS, HIGH_VOLATILITY_FIELDS
Cost Tracker	core/cost_tracker.py	Per-provider/stage token counts and USD pricing constants
Extractor	pipeline/stages/extractor.py	Active runtime; heuristic chunk scoring; supersedes legacy extraction.py
Frontend	frontend/	Next.js 14 / React 18; TanStack Query polling; PDF export via @react-pdf/renderer
Verification	pipeline/stages/verification.py	Standalone confidence scoring; candidate for merge with conflict_adjudicator.py

07 Technology Stack

Layer	Technology	Justification
Backend API	FastAPI 0.111	Async-first; native background tasks; automatic OpenAPI docs
Agent Orchestration	LangGraph	Node-based graph with typed state; supports conditional edges and mid-graph pause/resume
AI — Extraction	Google Gemini 2.5 Flash	Large context window (1M tokens); low cost per token for high-volume chunk extraction
AI — Debate	Groq llama-3.3-70b	Low-latency inference; free-tier key pool spreads rate limits across burst periods
Web Search	Tavily Search API	Purpose-built for LLM agents; returns pre-filtered results with source scores
Web Scraping	Firecrawl	Handles JS-rendered pages; returns clean markdown; async batch support
App Ratings	Google Play + iTunes	Official store APIs for live app rating data; no scraping required
Database	SQLite 3 + WAL	Zero-ops deployment; WAL journal mode gives adequate read/write concurrency
Data Validation	Pydantic v2	Runtime type validation for all agent state, API request/response, and schema models
Frontend	Next.js 14 / React 18	App Router; server components; TanStack Query for polling; Tailwind CSS
HTTP Client	httpx + aiohttp	Async HTTP for enrichment and app rating fetches
Process Comms	threading.Event + Lock	Clarification and cache-hit pauses; STORE guarded by STORE_LOCK
Environment	python-dotenv	API key injection with reject_placeholders=True guard
Logging	Python logging + FileHandler	Per-component log files: backend.log, debate_debug.log, frontend.log

08 Database & Data Models

8.1 Persistence Layer

Table	Purpose
run_snapshots	Cache of completed runs keyed by program_name_normalized; stores full AgentState as JSON for instant replay
runs	Run history index: run_id, mode, status, data_quality, run_state_json — survives process restarts
program_identities	Resolved ProgramIdentity records: name, brand, domain, country, confidence, status
sources	URL-level metadata: canonical_url, source_type, authority_score, fetched_at, http_status
pages	Cleaned page text per URL: title, cleaned_text, token_count, sanitizer_flags

Table	Purpose
chunks	Text chunks from pages: chunk_index, text, token_count, embedding_hash (forward-compatible)
claims	Extracted field claims: field_path, value_json, status, confidence, source_url, access_date
conflicts	Conflict records: field_path, strategy used, resolution status, judge reasoning, confidence
briefs	Final brief output per run: text, word_count, cited claim IDs, entailment flag
conversations	Q&A; turn log per run: question, answer, citations, timestamp
raw_documents	Post-Firecrawl documents keyed by url_hash: markdown content, source_authority, metadata
normalized_packets	Normalised extraction packets: identity_hash × source_url × chunk_id → structured fields

connect() opens with PRAGMA journal_mode=WAL and 5s busy timeout. **checkpoint()** runs PRAGMA wal_checkpoint(TRUNCATE) on FastAPI shutdown so committed data survives even if the -wal/-shm files are lost.

8.2 Schema Field Categories

program_basics

program_name · brand · industry · program_type · geography · membership_count · ownership_or_parent_company · launch_or_rebrand_history

earn_mechanics

base_earn_rate · earn_rate_unit · bonus_categories · co_brand_card_earn · partner_earn · non_transactional_earn · earning_exclusions

burn_mechanics

redemption_options · redemption_thresholds · point_value_cpp · cash_equivalent_value · expiry_policy · blackout_or_capacity_rules · transfer_options

tier_system

tier_names · qualification_criteria · tier_thresholds · qualification_period · tier_benefits · soft_landing_or_status_match · elite_bonus

partnerships

partner_names · partnership_type · details · partner_category · earn_details · burn_details · transfer_ratios · discontinued_partners

digital_experience

mobile_app_available · app_ratings · app_store_rating · play_store_rating · personalization_features · gamification_features · digital_wallet_or_card_linking · app_pain_points

member_sentiment

ratings · common_praise · common_complaints · complaint_frequency · sources_checked · review_sources_checked · forum_sources_checked · sentiment_summary

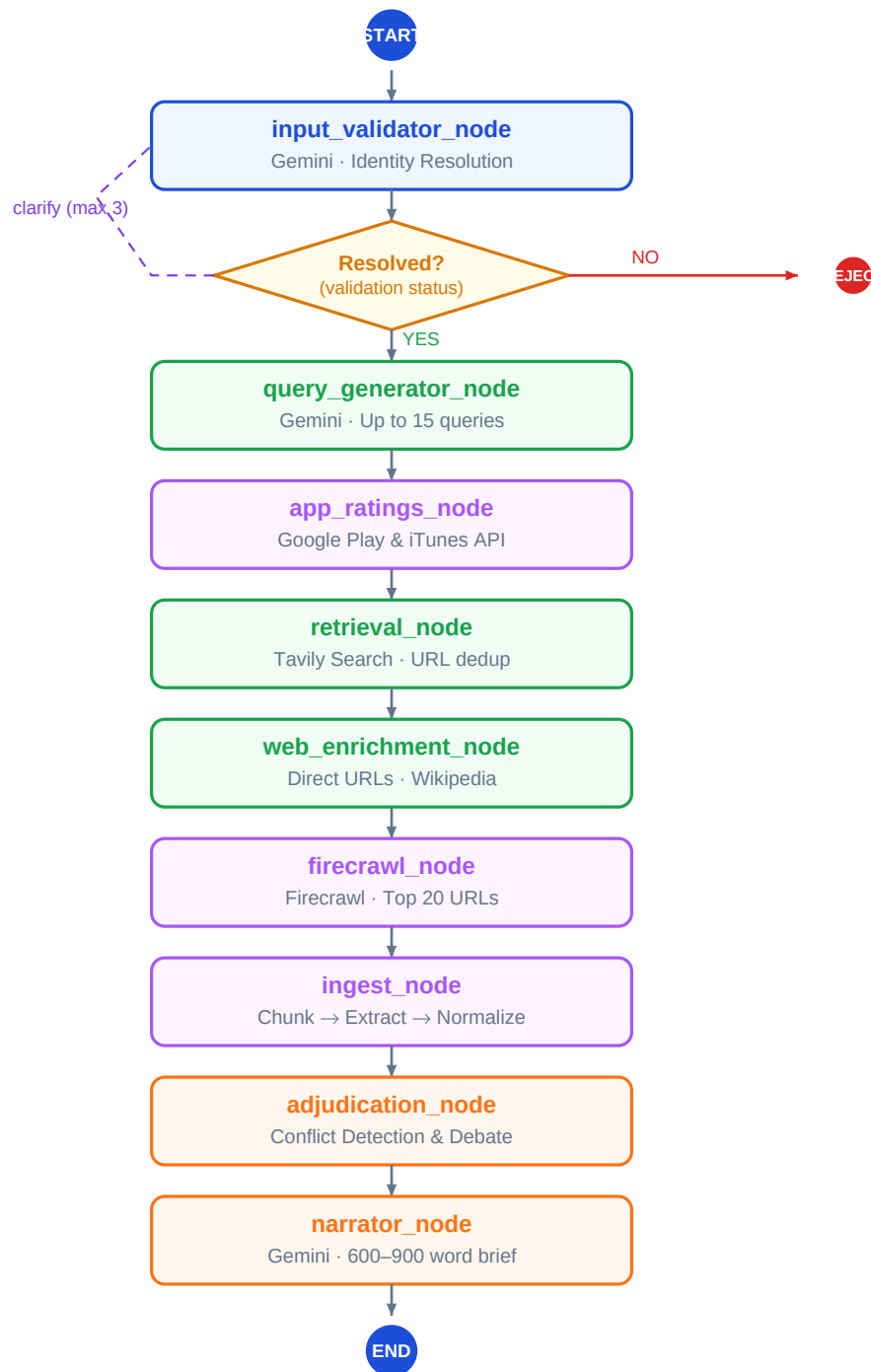
competitive_position

key_differentiators · weaknesses · closest_competitors

High-volatility fields (recency-aware adjudication): base_earn_rate · tier_thresholds · redemption_thresholds · point_value_cpp · app_ratings · recent_changes_last_6_months

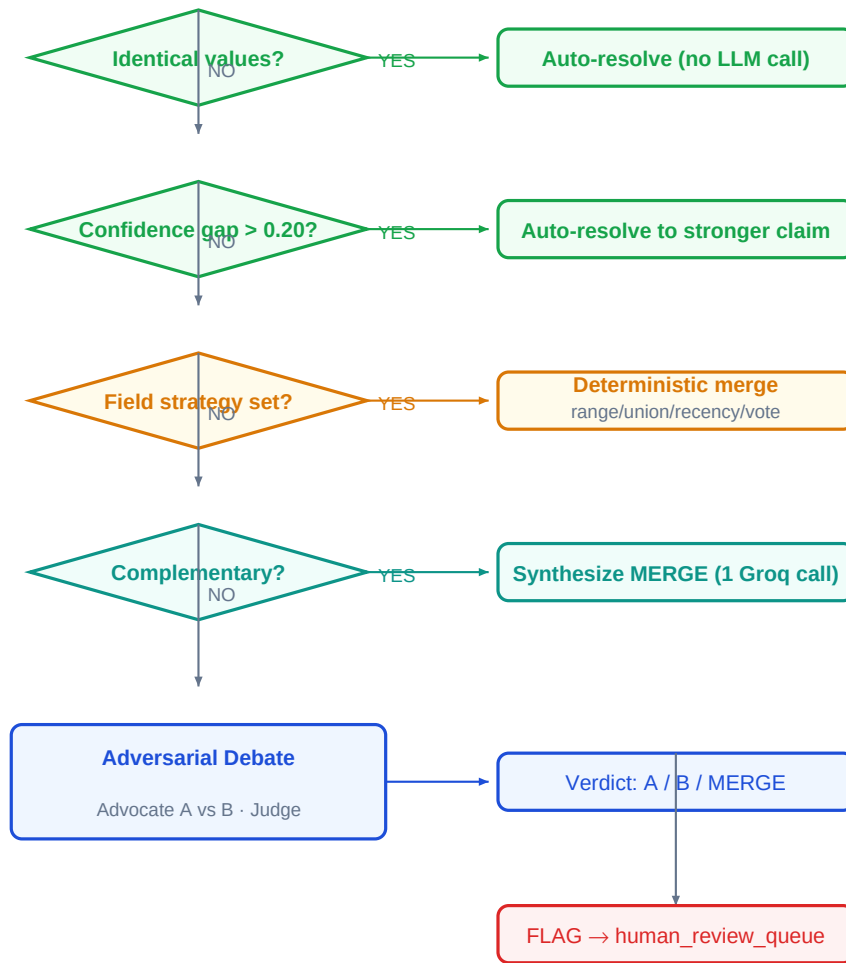
09 AI Agent Pipeline

9.1 Pipeline Flowchart



9.2 Conflict Resolution Decision Tree

5-Step Conflict Resolution Ladder



10 External Integrations

Google Gemini 2.5 Flash

Used by: validation.py · query_generator.py · extractor.py · narration.py

Purpose: Identity resolution, query planning, field extraction, brief narration. Large context window (1M tokens).

Config: GEMINI_API_KEY (overridable per stage: INPUT_VERIFIER_*, QUERY_GENERATOR_*, EXTRACTION_*, NARRATION_*)

Groq (llama-3.3-70b-versatile)

Used by: pipeline/adjudication/debate_engine.py + converse.py

Purpose: Low-latency adversarial debate (advocate A, B, judge) and interactive grounded Q&A.

Config: GROQ_API_KEYS (round-robin pool); 401 auto-eviction via _remove_client_from_pool

Tavily Search API

Used by: pipeline/stages/retrieval.py

Purpose: Query-driven URL discovery. 5 results/query, scored and typed. Purpose-built for LLM agents.

Config: TAVILY_API_KEY / TAVILY_API_BASE

Firecrawl

Used by: pipeline/stages/firecrawl_scraper.py

Purpose: Full-page markdown/PDF extraction of JS-rendered pages. Top 20 URLs per run. Partial failure tolerant.

Config: FIRECRAWL_API_KEY / FIRECRAWL_API_BASE

Google Play Scraper + iTunes

Used by: pipeline/stages/app_ratings_fetcher.py

Purpose: Live app ratings prefetched before Tavily retrieval begins. Bypasses scrape budget. Structured data.

Config: (no key required – public APIs)

Wikipedia REST API

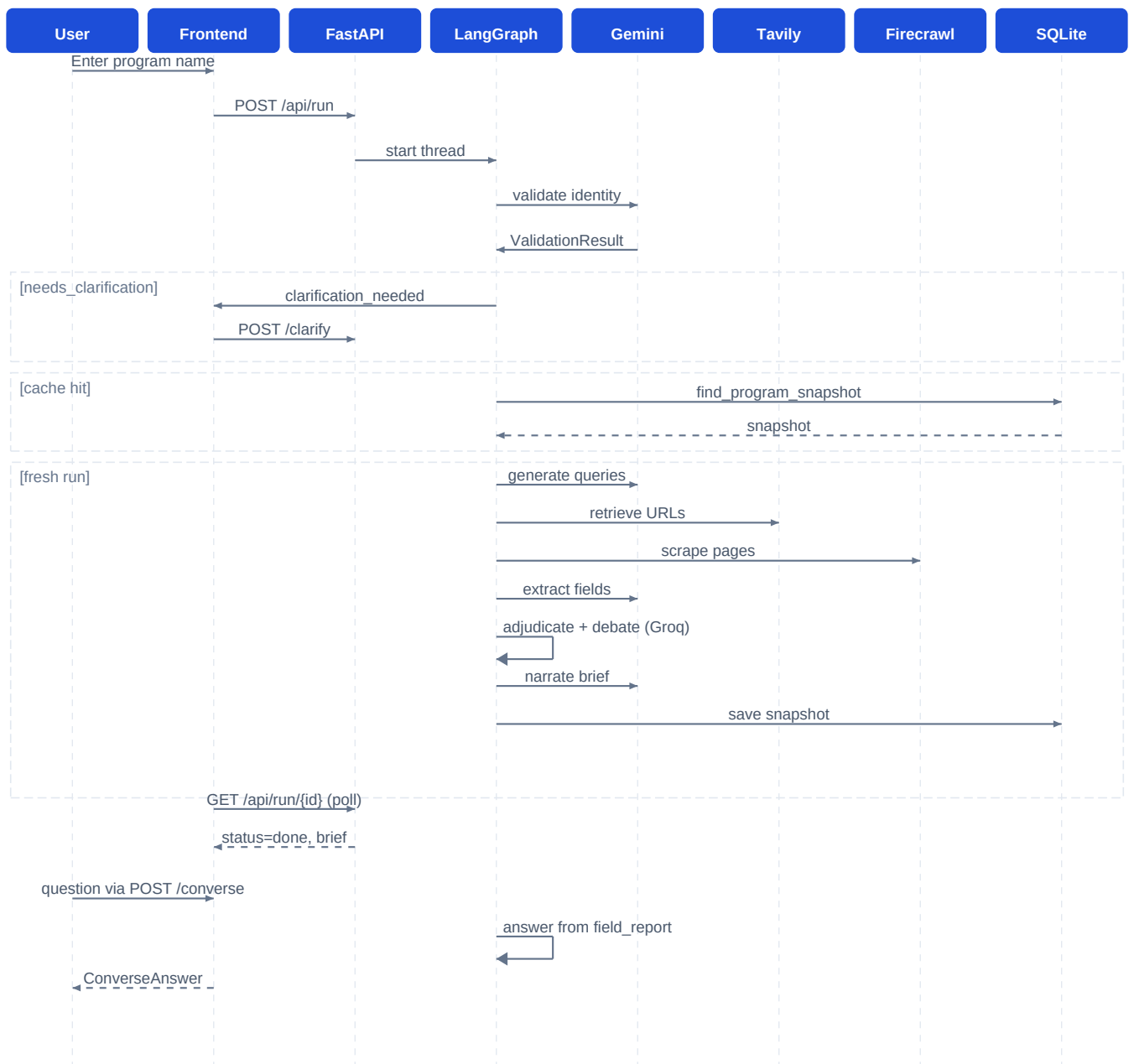
Used by: pipeline/stages/wikipedia_fetcher.py

Purpose: Free company/brand summary injected as synthetic evidence block. Zero search or scrape budget.

Config: (no key required – public API)

No vector database — chunk selection uses heuristic scoring (extractor.py). ChunkRef.embedding_hash exists in schema/DDDL as a forward-compatible migration path.

11 Sequence Diagram



12 Design Decisions & Trade-offs

Decision	Rationale	Trade-off
LangGraph invoked directly by server.py	Enables per-stage UI updates, clarification/cache pause, cancellation support	Two execution paths must stay behaviourally consistent
Thread-per-run	Simple to reason about; each RunRecord owns its own Lock, Events, stop_event	Does not scale horizontally beyond a single process
In-memory STORE + SQLite snapshot	Low-latency polling from memory; completed runs persist across restarts	Any run still running when the process restarts is lost

Decision	Rationale	Trade-off
Cheapest-sufficient conflict ladder	Full debate costs 3–5 LLM calls/conflict; most conflicts resolve deterministically	Requires maintaining FIELD_STRATEGY_MAP by hand; wrong strategy suppresses a real debate
Debate advocates see only structured metadata	Prevents re-hallucination of training-data facts; keeps arguments auditable	Quality bounded by how well metadata captures the actual disagreement
Gemini for extraction, Groq for debate	Splits cost/latency profiles: Gemini cheap for per-chunk extraction; Groq fast for debate	Two provider integrations; two sets of rate limits and keys
Round-robin Groq key pool with 401 eviction	Free-tier limits crash under concurrent debate bursts; spreading across keys prevents stalling	Pool management complexity; pool must be reset per run to avoid stale event-loop binding
No vector DB	Query-plan-driven Tavily + heuristic scoring sufficient for the fixed schema; kept system dependency-light	Can miss relevant text that semantic search would surface; embedding_hash unused
SQLite over client-server DB	Zero-ops single-node deployment; WAL gives adequate concurrency	Not suitable for multi-instance deployment; WAL journal files are a known failure point

13 Scalability, Reliability, Security & Observability

Scalability

- Single-process / single-node: STORE is an in-memory dict guarded by STORE_LOCK
- Thread-per-run; compare-mode runs programs sequentially
- Horizontal scaling path: externalize STORE to Redis, move execution to Celery/RQ

Reliability

- checkpoint() runs PRAGMA wal_checkpoint(TRUNCATE) on FastAPI shutdown
- server.sh integrity check on startup; corrupt DB moved to db_corrupted_backup/
- Per-program caching via run_snapshots survives process restarts
- All nodes catch exceptions → PipelineError → UI error (no run crash)
- Clarification/cache waits bounded to 300s via threading.Event.wait

Security

- API keys from .env via python-dotenv; reject_placeholders=True rejects "your_..." values
- CORS restricted to localhost:3000/3001 and 127.0.0.1 only
- No authentication layer on the API (gap before non-local deployment)
- Extraction and debate prompts include "hallucination fence" grounding instructions

Observability

- CostLedger: per-provider/stage token counts → USD via hard-coded pricing constants
- UI_STAGES tracked per run; surfaced to frontend via polling response
- logs/backend.log, logs/frontend.log, logs/debate_debug.log (FileHandler in debate_engine.py)
- Per-stage cost breakdown visible in the frontend cost panel

14 Error Handling & Retry Strategy

Node-level isolation

Every graph node catches exceptions internally and writes a PipelineError delta. The background thread continues without crashing; the UI reflects an error state for that stage.

Firecrawl partial failure

The scraper continues if individual URL fetches fail; the node only raises if zero URLs succeed overall.

Groq rate limiting

debate_engine.py::call_groq retries up to $\text{pool_size} \times 2$ attempts, rotates keys on HTTP 429, permanently evicts keys on HTTP 401 via `_remove_client_from_pool`.

Judge output parsing

parse_judge_output tolerates malformed or partial JSON, falling back to FLAG_FALLBACK_REASONING so the conflict always has a recorded outcome.

Non-fatal enrichment failures

web_enrichment_node treats URL-seeding and Wikipedia fetch errors as non-fatal; logging warnings and continuing with available evidence.

User-triggered cancellation

record.stop_event checked before every stage transition; in-flight stages marked error, run_status set to cancelled.

Debate engine fallback

Unhandled exceptions from the debate engine produce a FLAG with deciding_factor: "unresolvable" and confidence 0.40.

15 Future Enhancements

1 Embedding-based Retrieval

ChunkRef.embedding_hash and chunks table schema are forward-compatible; moving from heuristic scoring to semantic vector search improves extraction recall.

2 Multi-instance Scaling

Externalise STORE to Redis and move run execution to Celery or RQ.

3 Authentication & Authorisation

No auth layer on the API; required before non-local deployment. Candidate: OAuth2/OIDC middleware on FastAPI.

4 Streaming Updates via WebSocket / SSE

Replace polling (GET /api/run/{id}) with WebSocket or SSE channel for lower-latency stage-progress updates.

5 Crash-resumable Runs

Persist AgentState to the database after each completed node to enable mid-run restart recovery.

6 verification.py Consolidation

Standalone confidence-scoring module is a candidate for merge with conflict_adjudicator.py to reduce overlapping scoring paths.

16 Conclusion

Kobie demonstrates that a rigorous, source-grounded intelligence pipeline can be built from commodity LLM APIs and open-source tooling without a vector database or bespoke model. Its design is governed by a single architectural constraint: "Never hallucinate a fact."

Every extracted value carries a `source_url`. Conflicts between sources are resolved through an auditable ladder of deterministic strategies — using the cheapest sufficient method — before falling back to metadata-only adversarial debate. The narrative brief and the grounded Q&A interface are both constrained to adjudicated evidence only; the LLM is used as a synthesiser and writer, not as a knowledge store.

The result is an agent whose outputs can be independently verified: every claim traces back to a retrieved page, and every conflict resolution decision is recorded with its strategy, judge reasoning, and confidence score. This auditability is the architectural property that distinguishes Kobie from a simple chatbot wrapper over a search API.

FastAPI

LangGraph

Gemini 2.5 Flash

Groq

Tavily

Firecrawl

SQLite WAL

Next.js 14

TanStack Query

Pydantic v2